

SIMATS ENGINEERING

ASSESSMENT TOOL 1

COURSE NAME:CSA0408

COURSE CODE: Operating Systems

COURSE OUTCOME COVERED: co4: *Optimize various file allocation strategies and disk scheduling algorithms to identify optimal methods for enhancing system performance.*

Assessment Tool Weightage: 30%

Course Outcome 1 Weightage: 20%

Dr.J.Velmurugan / Professor
Dept. of CSE

QUESTIONS: 5

Total Marks:25

Annexure A

CODING CHALLENGE/HACKATHON

Code of Conduct:

I D.NAVEEN KUMAR(192511401) (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.

I understand that any violation of academic integrity rules will result in disciplinary action.

D.NAVEEN KUMAR

Signature of the student:

Annexure A

CODING CHALLENGE

S.No	Coding Challenge Questions	Marks
1	<u>Write a program to simulate the three file allocation methods—Contiguous, Linked, and Indexed. The program should accept inputs including the total disk size (number of blocks), the file size(number of blocks required),and the allocation method (1forContiguous, 2for Linked, 3 for Indexed). Based on the selected method, the program must allocate disk blocks accordingly, display the allocated block numbers, and handle fragmentation scenarios, particularly when contiguous allocation fails due to insufficient continuous space. The output should show the list of allocated blocks or an appropriate error message if allocation is not possible. Ensure that built-in memory allocation functions are not used, and assume that disk blocks are numbered starting from 0.</u>	5
2	<u>Develop a program to manage free disk space using a bitmap technique. The program should accept inputs including the total number of disk blocks, the initial bitmap (where 0 represents free and 1 represents allocated), and a request for allocation specifying the number of blocks required. The program must identify and allocate free blocks efficiently, update the bitmap after allocation, and also support</u>	5

	<u>deallocation of specified blocks with appropriate bitmap updates. The output should display the updated bitmap after allocation and deallocation operations. Ensure that the program efficiently searches for free blocks and properly handles Cases where sufficient space is not available.</u>	
<u>3</u>	<u>Implement a program that simulates at least three disk scheduling algorithms: FCFS, SSTF, and SCAN. The program should take as input the disk request queue, the initial head position, and the disk size. It must compute the total seek time for each algorithm, display the order in which disk requests are serviced, and compare their performance. The output should include the seek sequence and total seek time for each algorithm, along with identification of the best-Performing algorithm based on minimum seek time.</u>	<u>5</u>
<u>4</u>	<u>Write a program to implement the C-SCAN and LOOK disk scheduling algorithms. The program should accept inputs including the request queue, the initial head position, and the direction of head movement (left or right). It must simulate both algorithms, calculate the total head movement required to service all requests, and compare their efficiency. The output should display the seek sequence, total head movement, and a conclusion indicating which algorithm performs better along with a brief justification. Optionally, the program may include a graphical visualization of head movement.</u>	<u>5</u>
<u>5</u>	<u>Design a program that integrates file allocation strategies with disk scheduling techniques to optimize system performance. The program should accept inputs such as the total number of disk blocks, file size (number of blocks required), disk request queue, initial head position, allocation method (Contiguous or Indexed), and disk scheduling method (FCFS or SSTF). It must allocate disk blocks based on the chosen allocation strategy, simulate disk scheduling to access the allocated blocks, calculate the total seek time, and compare performance across different combinations. The output should display allocated blocks, seek sequence, total seek time, and identify the best combination of allocation and scheduling methods. Ensure the program handles allocation failures, avoids duplicate block allocation, and maintains efficient scheduling.</u>	<u>5</u>

1.Contiguous, Linked and Indexed File Allocation

```
#include<studio.h>
Int main(){
    int disk[100], n, size, method, i, start, count = 0;

    printf("Enter total disk blocks:
    "); scanf("%d", &n);

    for (i = 0; i < n;
        i++) disk[i] =
        0;

    printf("Enter
    file size: ");
    scanf("%d",
    &size);

    printf("Enter method (1-Contiguous, 2-Linked, 3-Indexed): ");
    scanf("%d", &method);

    if (method == 1) {
```

```

    for (i = 0; i <
        n; i++) { if
        (disk[i] ==
        0) {
            count++;
            if (count ==
                size) {
                start = i -
                size + 1;
                printf("Allocated
                blocks: "); for (int j
                = start; j <= i; j++)
                    printf("
                    %d ", j);
                return 0;
            }
        } else
            count = 0;
    }
    printf("Contiguous allocation failed.\n");
}

else if (method == 2)
{
    printf("Allocated
    blocks: ");
    for (i = 0; i < n && i < size;
        i++) printf("%d ", i);
}

else if (method == 3)
{ printf("Index
    block: 0\n");
    printf("Allocated
    blocks: ");
    for (i = 1; i <= size && i <
        n; i++) printf("%d ", i);
}

else

    printf("Invalid method.");

    return 0;
}

```

Input:

Enter total disk
blocks: 10 Enter
file size: 4

Enter method (1-Contiguous, 2-Linked, 3-

Indexed): 1 Output:

Allocated blocks: 0 1 2 3

2.Bitmap Free Space Management

```
#include<studio.h>
```

```
Int main(){
```

```
    int n, bitmap[100], req, i, count = 0, block;
```

```
    printf("Enter number of disk  
blocks: "); scanf("%d", &n);
```

```
    printf("Enter bitmap (0-free, 1-  
allocated):\n"); for (i = 0; i < n; i++)  
        scanf("%d", &bitmap[i]);
```

```
    printf("Enter number of blocks to allocate:  
"); scanf("%d", &req);
```

```
    for (i = 0; i < n && count < req; i++) { if (bitmap[i] ==  
0) {  
        bitmap[i]  
        = 1;  
        count++;  
    }  
}
```

```
    if (count < req) {  
        printf("Not enough free  
blocks.\n"); return 0;  
    }
```

```
    printf("Bitmap after  
allocation:\n"); for (i = 0; i  
< n; i++)  
        printf("%d ", bitmap[i]);
```

```
    printf("\nEnter block to  
deallocate: "); scanf("%d",  
&block);
```

```
    if (block >= 0 &&  
        block < n)
```

```

        bitmap[block] =
        0;

    printf("Bitmap after
    deallocation:\n"); for (i = 0; i
    < n; i++)
        printf("%d ", bitmap[i]);

    return 0;
}

```

Input:

Enter number of disk
blocks: 8 Enter bitmap (0-
free, 1-allocated):
1 0 0 1 0 1 0 0
Enter number of blocks to
allocate: 2 Enter block to
deallocate: 3

Output:

Bitmap after allocation:
1 1 1 1 0 1 0 0

Bitmap after deallocation:
1 1 1 0 0 1 0 0

3.FCFS, SSTF and SCAN Disk Scheduling

```

#include
<stdio.h>
#include
<stdlib.h>

int main() {
    int a[50], n, head, size, i, j, total, pos, temp;

    printf("Enter disk
    size: "); scanf("%d",
    &size);

    printf("Enter number of
    requests: "); scanf("%d", &n);

    printf("Enter request
    queue:\n"); for (i = 0; i
    < n; i++)
        scanf("%d", &a[i]);

```

```
printf("Enter initial head
position: "); scanf("%d",
&head);
```

```
/*
F
C
FS
*/
```

```
tot
al
=
```

```
0;
```

```
po
```

```
s
```

```
=
```

```
he
```

```
ad
```

```
;
```

```
printf("\nFCFS: %d
```

```
", head); for (i = 0; i
```

```
< n; i++) {
```

```
total +=
```

```
abs(pos
```

```
- a[i]);
```

```
pos =
```

```
a[i];
```

```
printf("-> %d ", a[i]);
```

```
}
```

```
printf("\nTotal Seek Time = %d\n", total);
```

```
/* SSTF */
```

```
int visited[50]
```

```
= {0}; total =
```

```
0;
```

```
pos = head;
```

```
printf("\nSSTF: %d
```

```
", head); for (i = 0; i
```

```
< n; i++) {
```

```
int min = 9999, index = -1;
```

```

    for (j = 0; j < n; j++) {
        if (!visited[j] && abs(pos - a[j]) <
            min) { min = abs(pos - a[j]);
                index = j;
            }
    }

    visited[index] = 1; total
    += min;
    pos =
    a[index];
    printf(
    "->
    %d ",
    pos);
}

printf("\nTotal Seek Time = %d\n", total);

/* SCAN -
move right */
total = 0;
pos = head;

for (i = 0; i < n; i++)
    for (j = i + 1; j
        < n; j++) if
        (a[i] > a[j])
        {
            te
            m
            p =
            a[i
            ];
            a[i
            ] =
            a[j
            ];
            a[j
            ] =
            te
            m
            p;
        }
    printf("\nSCAN: %d
    ", head); for (i = 0; i
    < n; i++) {
        if (a[i] >= head) {

```

```

        total += abs(pos
        - a[i]); pos =
        a[i];
        printf("-> %d ", a[i]);

    }
}

total += abs(pos -
(size - 1)); pos = size
- 1;
printf("-> %d ", pos);

for (i = n - 1; i
    >= 0; i--) { if
    (a[i] < head) {
        total += abs(pos
        - a[i]); pos =
        a[i];
        printf("-> %d ", a[i]);
    }
}

printf("\nTotal Seek Time = %d\n", total);

return 0;
}

```

Input:

Enter disk size: 200

Enter number of

requests: 8 Enter

request queue:

98 183 37 122 14 124 65 67

Enter initial head position: 53

Output:

FCFS: 53 -> 98 -> 183 -> 37 -> 122 -> 14 -> 124 -> 65 -> 67

Total Seek Time = 640

SSTF: 53 -> 65 -> 67 -> 37 -> 14 -> 98 -> 122 -> 124 -> 183

Total Seek Time = 236

SCAN: 53 -> 65 -> 67 -> 98 -> 122 -> 124 -> 183 -> 199 -> 37 -> 14

Total Seek Time = 331

4.C-SCAN and LOOK Disk Scheduling

```

#include
<stdio.h>

```



```

#include
<stdlib.h>

int main() {
    int a[50], n, head, size,
    direction; int i, j, pos,
    total, temp;

    printf("Enter disk
    size: "); scanf("%d",
    &size);

    printf("Enter number of
    requests: "); scanf("%d", &n);

    printf("Enter request
    queue:\n"); for (i = 0; i
    < n; i++)
        scanf("%d", &a[i]);

    printf("Enter initial head
    position: "); scanf("%d",
    &head);

    printf("Enter direction (1-Right, 2-Left): ");
    scanf("%d", &direction);

    for (i = 0; i < n; i++)
        for (j = i + 1; j
        < n; j++) if
        (a[i] > a[j])
        {
            te
            m
            p =
            a[i
            ];
            a[i
            ] =
            a[j
            ];
            a[j
            ] =
            te
            m
            p;
        }

    /* C-
    SCAN

```

```

*/ total
= 0;
pos = head;
printf("\nC-SCAN: %d ",
head); if

(direction ==

1) { for (i = 0;

i < n; i++)

    if (a[i] >=
        head) {
        total +=
        abs(pos
        - a[i]);
        pos =
        a[i];
        printf("-> %d ", pos);
    }

    total += abs(pos -
(size - 1)); pos = size
- 1;
    printf("-> %d ", pos);

    total +=
    size - 1;
    pos = 0;
    printf("-> 0 ");

    for (i = 0; i
        < n; i++)
        if (a[i] <
            head) {
            total +=
            abs(pos
            - a[i]);
            pos =
            a[i];
            printf("-> %d ", pos);
        }
}

printf("\nC-SCAN Total = %d\n", total);

```

```

/*
L
O
O
K
*/
total
=
0;
pos
=
head
ad
;
printf("\nLOOK: %d
", head); if (direction
== 1) {
    for (i = 0; i
        < n; i++)
        if (a[i]
            >= head)
            {
                total +=
                abs(pos
                - a[i]);
                pos =
                a[i];
                printf("-> %d ", pos);
            }

    for (i = n - 1; i >= 0; i--)
        if (a[i] <
            head) {
                total +=
                abs(pos
                - a[i]);
                pos =
                a[i];
                printf("-> %d ", pos);
            }
}

printf("\nLOOK Total = %d\n", total);

return 0;
}
Input:

```

Enter disk size: 200
Enter number of
requests: 8 Enter
request queue:
98 183 37 122 14 124 65 67
Enter initial head
position: 53 Enter
direction (1-Right, 2-
Left):
1

Output:

C-SCAN: 53 -> 65 -> 67 -> 98 -> 122 -> 124 -> 183 -> 199 -> 0 -> 14 ->
37 C-SCAN Total = 331

LOOK: 53 -> 65 -> 67 -> 98 -> 122 -> 124 -> 183 -> 37 -> 14
LOOK Total = 299

5.File Allocation + Disk Scheduling

```
#include
<stdio.h>
#include
<stdlib.h>

int main() {
    int n, size, method,
    schedule, head; int
    blocks[100], i, j, pos, total,
    temp;

    printf("Enter total disk blocks:
    "); scanf("%d", &n);

    printf("Enter
    file size: ");
    scanf("%d",
    &size);

    printf("Allocation (1-Contiguous, 2-
    Indexed): "); scanf("%d", &method);

    printf("Scheduling (1-FCFS, 2-SSTF): ");
    scanf("%d", &schedule);

    printf("Enter initial head
    position: "); scanf("%d",
    &head);
```

```

if (size > n) {
    printf("Allocation
    failed.\n");
    return 0;
}
printf("\nAllocated

blocks: "); if

(method == 1) {

    for (i = 0; i < size; i++) {
        blocks
        [i]
        = i;
        printf(
        "%d ",
        blocks[i]);
    }
} else {
    printf("Index
    Block = 0, "); for
    (i = 0; i < size;
    i++) {
        blocks[i] = i +
        1; printf("%d ",
        blocks[i]);
    }
}

total
=
0;
positions
=
head
ad
;
printf("\n\nSeek Sequence: %d
", head); if (schedule == 1) {
    for (i = 0; i < size; i++) {

```

```

        total += abs(pos -
        blocks[i]); pos =
        blocks[i];
        printf("-> %d ", pos);
    }
} else {
    int visited[100] = {0};

    for (i = 0; i < size; i++) {
        int min = 9999, index = -1;

        for (j = 0; j < size; j++) {
            if (!visited[j] && abs(pos - blocks[j]) <
            min) { min = abs(pos - blocks[j]);
            index = j;
            }
        }

        visited[index] = 1;
        total +=
        min;
        pos =
        blocks[index];
        printf("->
        %d ", pos);
    }
}

printf("\nTotal Seek Time = %d\n", total);

return 0;
}

```

Input:

Enter total disk

blocks: 20 Enter

file size: 5

Allocation (1-Contiguous, 2-

Indexed): 1 Scheduling (1-

FCFS, 2-SSTF): 2

Enter initial head position: 10

Output:

Allocated blocks: 0 1 2 3 4

Seek Sequence: 10 -> 4 -> 3 -> 2

-> 1 -> 0 Total Seek Time = 10

RUBRICS FOR CALCULATION

Criteria	Weightage (%)	Level 4 (Exemplary)	Level 3 (Proficient)	Level 2 (Developing)	Level 1 (Beginning)	Total
Conceptual Understanding	30%					
Accuracy of Answers	25%					
Application of Knowledge	20%					
Time Management	15%					
Question Interpretation	10%					
Total						